

Relatório de Mapeamento NoSQL - Parte 2

Discentes:

João Gustavo Costa Bittencourt Santos

João Vitor Vital Leão

Lucca Pedreira Dultra

1. Decisões Arquiteturais e de Mapeamento

A migração do modelo relacional original para o MongoDB orientou-se pelas seguintes decisões de projeto:

- **Abordagem Normalizada (1:1):** Optamos por não utilizar documentos embutidos (*embedded documents*) de forma excessiva. Cada uma das 16 tabelas relacionais foi convertida em uma coleção independente. Essa estratégia evita anomalias de atualização e duplicação de dados, mantendo o ecossistema fiel às regras de negócio da universidade.
- **Garantia de Restrições via Aplicação:** Sendo o MongoDB *schemaless*, a integridade referencial (Chaves Estrangeiras), as restrições de chave (Primary Key/Unique) e domínio (Not Null) foram delegadas à camada de aplicação (Python).
- **Tipagem BSON/JSON:** Dados multivalorados no relacional, como telefone e email, foram mapeados nativamente como *Arrays* (vetores) no NoSQL, aproveitando a flexibilidade do formato JSON.

2. Mapeamento das Coleções

Coleção: usuario

- **Decisão de Mapeamento:** Coleção base que centraliza os dados de acesso e contato. Optamos por usar Arrays nativos para email e telefone, eliminando a necessidade de tabelas associativas e utilizando a flexibilidade do NoSQL.
- **Script do Esquema:**

```
{
  cpf: Number,      // Primary Key
  nome: String,     // Not Null
  data_nascimento: Date,
  email: Array,
```

```
telefone: Array,  
login: String,      // Unique  
senha: String  
}
```

- **Documento de Exemplo:**

```
{ "cpf": 11111111100, "nome": "Prof A", "data_nascimento": "1980/03/05", "email":  
["profA@email.com"], "telefone": ["99998888", "88889999"], "login": "profa", "senha": "senha1" }
```

Coleção: estudante

- **Decisão de Mapeamento:** Mantida separada de usuario para registrar métricas acadêmicas. A integridade com usuario é validada pelo sistema Python no momento da inserção.
- **Script do Esquema:**

```
{  
  mat_estudante: String, // Primary Key  
  cpf: Number,           // Referência para 'usuario' (Unique)  
  MC: Number,  
  ano_ingresso: Int  
}
```

- **Documento de Exemplo:**

```
{ "mat_estudante": "E101", "cpf": 22222222201, "MC": 7.0, "ano_ingresso": 2021 }
```

Coleção: professor

- **Decisão de Mapeamento:** Estrutura espelhada ao estudante. O tipo tipo_jornada_trabalho e formacao (Enums no PostgreSQL) são validados na aplicação.
- **Script do Esquema:**

```
{  
  mat_professor: String, // Primary Key  
  cpf: Number,           // Referência para 'usuario' (Unique)  
  departamento: String, // Referência para 'departamento'  
  formacao: String,  
  data_admissao: Date,  
  tipo_jornada_trabalho: String,  
  salario: Number  
}
```

- **Documento de Exemplo:**

```
{ "mat_professor": "P100", "cpf": 11111111100, "departamento": "DCOMP", "formacao":  
"Doutorado", "data_admissao": null, "tipo_jornada_trabalho": "20h", "salario": 2000 }
```

Coleção: departamento

- **Decisão de Mapeamento:** Coleção que administra orçamentos e chefias.
- **Script do Esquema:**

```
{  
  cod_depto: String,    // Primary Key  
  nome: String,        // Not Null  
  chefe: String,        // Referência para 'professor'  
  orcamento: Number,  
  comissal: Number  
}
```

- **Documento de Exemplo:**

```
{ "cod_depto": "DCOMP", "nome": "Departamento de Computação", "chefe": "P100", "orcamento":  
10000, "comissal": 1000 }
```

Coleção: curso

- **Decisão de Mapeamento:** O ID autoincremental (SERIAL) do SQL foi substituído pelo padrão `_id` numérico no MongoDB para facilitar as amarrações do CRUD.
- **Script do Esquema:**

```
{  
  _id: Int,             // Primary Key  
  nome: String,         // Not Null  
  grau: String,  
  turno: String,        // Not Null  
  campus: String,  
  nivel: String  
}
```

- **Documento de Exemplo:**

```
{ "_id": 1, "nome": "Ciência da Computação", "grau": "Bacharelado", "turno": "Vespertino",  
"campus": "São Cristóvão", "nivel": "Graduação" }
```

Coleção: vinculo

- **Decisão de Mapeamento:** Tabela associativa lógica. Mantida isolada para evitar sobrecarga no documento do estudante caso ele tenha múltiplos vínculos históricos.
- **Script do Esquema:**

```
{
  _id: Int,
  mat_estudante: String, // Referência para 'estudante'
  curso: Int,           // Referência para 'curso'
  data_entrada: Date,
  status: String,
  data_saida: Date
}
```

- **Documento de Exemplo:**

```
{ "_id": 1, "mat_estudante": "E101", "curso": 3, "data_entrada": null, "status": "Ativo", "data_saida": null }
```

Coleção: disciplina

- **Decisão de Mapeamento:** Mapeamento direto com suporte a pré-requisitos lógicos apontando para a própria coleção.
- **Script do Esquema:**

```
{
  cod_disc: String,    // Primary Key
  nome: String,        // Not Null
  pre_req: String,     // Auto-referência
  credits: Int,
  depto_responsavel: String // Referência para 'departamento'
}
```

- **Documento de Exemplo:**

```
{ "cod_disc": "COMP0198", "nome": "Programação Orientada à Objetos", "pre_req": "COMP0197", "credits": 4, "depto_responsavel": "DCOMP" }
```

Coleção: semestre

- **Decisão de Mapeamento:** Mapeamento composto onde ano e semestre formavam a PK noSQL.
- **Script do Esquema:**

```
{
  ano: Int,
  semestre: Int,
  data_inicio: Date,
  data_fom: Date
}
```

- **Documento de Exemplo:**

```
{ "ano": 2017, "semestre": 2, "data_inicio": null, "data_fom": null }
```

Coleção: turma

- **Decisão de Mapeamento:** Utilização de `_id` numérico substituindo a *Sequence* do PostgreSQL.
- **Script do Esquema:**

```
{  
  _id: Int,           // Primary Key  
  cod_disc: String,   // Referência para 'disciplina'  
  numero: Int,  
  ano: Int,  
  semestre: Int  
}
```

- **Documento de Exemplo:**

```
{ "_id": 1, "cod_disc": "COMP0212", "numero": 1, "ano": 2017, "semestre": 2 }
```

Coleção: cursa

- **Decisão de Mapeamento:** Coleção que registra o desempenho. Mapeamento focado na associação aluno x turma.
- **Script do Esquema:**

```
{  
  mat_estudante: String, // Referência para 'estudante'  
  id_turma: Int,         // Referência para 'turma'  
  nota: Number  
}
```

- **Documento de Exemplo:**

```
{ "mat_estudante": "E112", "id_turma": 7, "nota": 4.1 }
```

Coleção: leciona

- **Decisão de Mapeamento:** Tabela associativa entre professores e turmas.
- **Script do Esquema:**

```
{  
  id_turma: Int,       // Referência para 'turma'  
  mat_professor: String // Referência para 'professor'  
}
```

- **Documento de Exemplo:**

```
{ "id_turma": 1, "mat_professor": "P100" }
```

Coleção: projeto

- **Decisão de Mapeamento:** Representa o catálogo de projetos de pesquisa da universidade. Optamos por mantê-la como coleção independente para facilitar a contagem e listagem isolada de projetos sem a necessidade de cruzar dados com os planos.
- **Script do Esquema:**

```
{  
  
  id_projeto: Int,    // Primary Key  
  
  descricao: String  
  
}
```

- **Documento de Exemplo:**

```
{ "id_projeto": 1, "descricao": "Projeto 1" }
```

Coleção: plano

- **Decisão de Mapeamento:** Tabela associativa que liga estudantes, professores e projetos. Mantida como coleção separada para documentar o histórico de orientações por ano, garantindo a integridade referencial via aplicação.
- **Script do Esquema:**

```
{  
  
  id_projeto: Int,    // Referência para 'projeto'  
  
  mat_professor: String, // Referência para 'professor'  
  
  mat_estudante: String, // Referência para 'estudante'  
  
  ano: Int  
  
}
```

- **Documento de Exemplo:**

```
{ "id_projeto": 1, "mat_professor": "P100", "mat_estudante": "E103", "ano": 2018 }
```

Coleções Físicas: sala, horario e alocacao

- **Decisão de Mapeamento:** Embora sem dados de inserção no dump original, as coleções foram modeladas no banco para manter 100% de paridade com o esquema lógico.
- **Script do Esquema (alocacao):**

```
{  
  id_turma: Int,  
  id_horario: Int,  
  id_sala: Int  
}
```

- **Documento de Exemplo:**

As coleções não possuem documentos no dump original disponibilizado para a atividade. Por esse motivo, não há documento de exemplo correspondente.